

ROBOTICS & COMPUTER VISION • LECTURE

Image Segmentation

*Teaching a machine to
understand a picture one pixel
at a time.*

Intuition → Architecture → Code → Evaluation



AGENDA

Today's roadmap

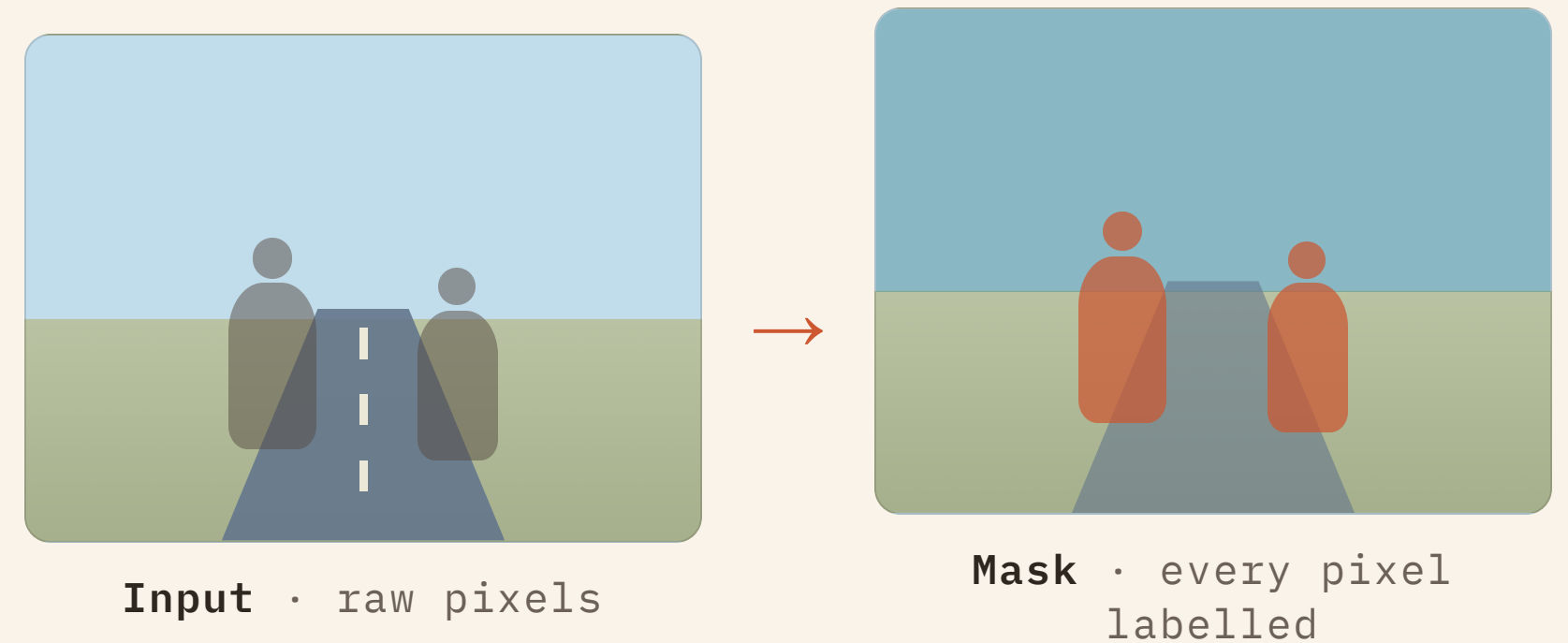
- 01 **The intuition** — what segmentation is, why it matters, and its three types
Classification vs. detection vs. segmentation · semantic, instance, panoptic
- 02 **The evolution** — from classical methods to deep learning and foundation models
Thresholding · encoder-decoder · FCN · U-Net · Segment Anything (SAM)
- 03 **The toolkit** — the Python computer-vision ecosystem and what a mask really is
OpenCV · PyTorch · Torchvision · Albumentations
- 04 **Hands-on** — a four-step inference pipeline you can run today
Load → model → inference → visualise
- 05 **Evaluation** — measuring quality with IoU and Dice, plus common pitfalls
Metrics · class imbalance · compute cost · annotation

What is image segmentation?

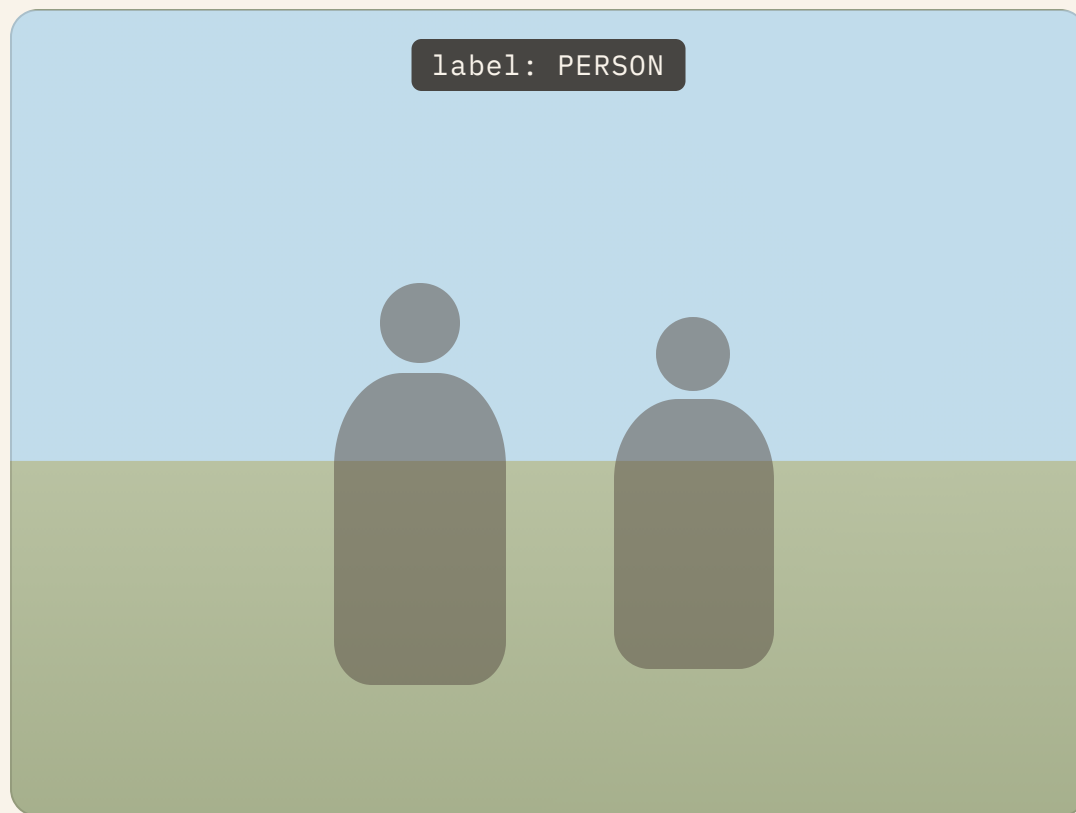
Segmentation assigns a **class label to every pixel** — turning a raw photo into a map of *what is where*.

The output is itself an image: same height and width as the input, but each pixel stores a category instead of a colour. We call this a **segmentation mask**.

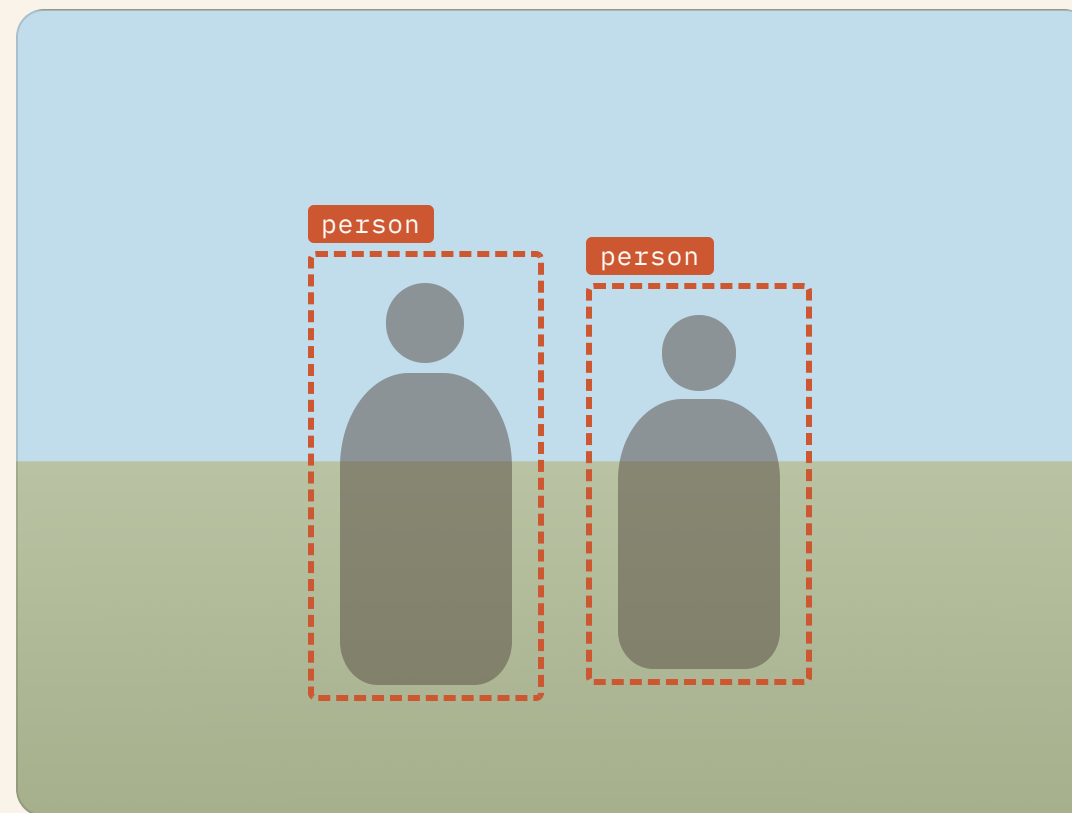
DENSE, PIXEL-LEVEL PREDICTION



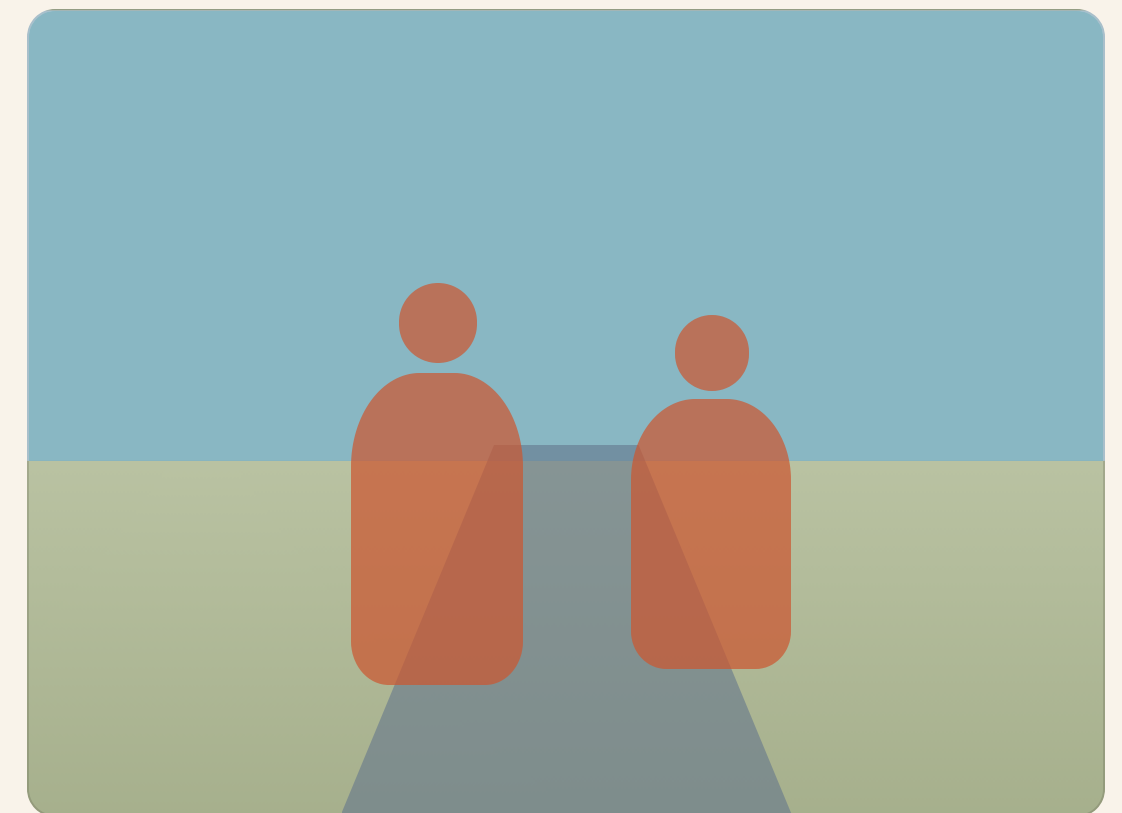
Three ways a machine can “see”



Classification – what is in it?
One label for the whole image. No location.



Detection – where, roughly?
Boxes give location & count, but include background.



Segmentation – exactly which pixels?
The precise silhouette of every object & region.

Why segmentation matters



Drop: street scene with road / lane masks

AUTONOMOUS ROBOTS

Drivable surface, kerbs & obstacles outlined exactly — the basis for safe **navigation**.



Drop: surgical frame (instruments / anatomy)

SURGICAL DATA SCIENCE

Instruments & anatomy segmented per frame — e.g. **cataract surgery** video.

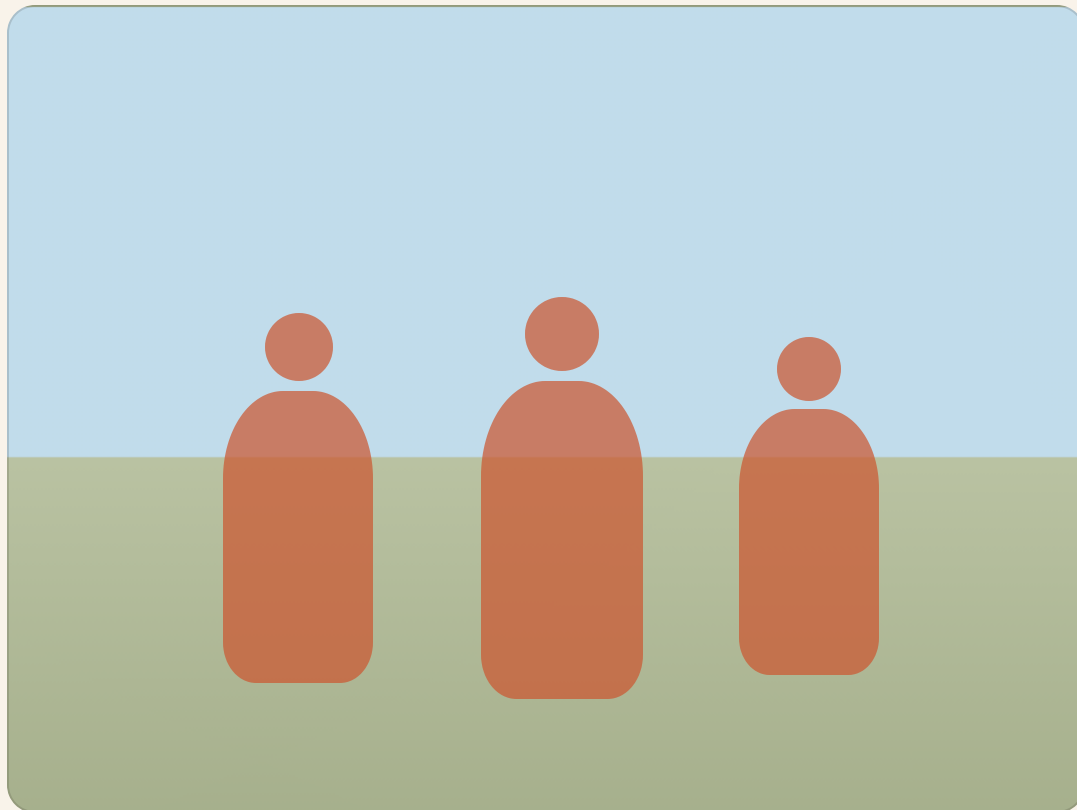


Drop: medical scan with tumour outline

MEDICAL IMAGING

Tumours & organs outlined to **measure volume** and track change over time.

Semantic, instance, and panoptic



Semantic

Every pixel gets a **class**. All people share one colour — no counting.



Instance

Each **object** separated — person 1, 2, 3 — but background ignored.



Panoptic

Both at once — class for *stuff*, identity for *things*. One label per pixel.

Classical methods: thresholding & edges

THRESHOLDING

Split by brightness

Pixels brighter than a cut-off → foreground, darker → background. **Otsu's method** finds the cut-off automatically.

```
mask = (gray > T)
```

EDGE DETECTION

Find sharp changes

Detectors like **Canny** mark pixels where intensity changes abruptly — the *gradient* — tracing object outlines.

```
cv2.Canny(img, 100, 200)
```

REGION GROWING

Grow from a seed

Start at a seed pixel, absorb neighbours with similar colour. Related: **watershed** & **k-means** on colour.

```
seed → expand if  $|\Delta| < \text{tol}$ 
```

All three use only **low-level cues** — brightness, colour, local contrast. Fast and training-free, but blind to *what an object actually is*.

Where classical methods break down

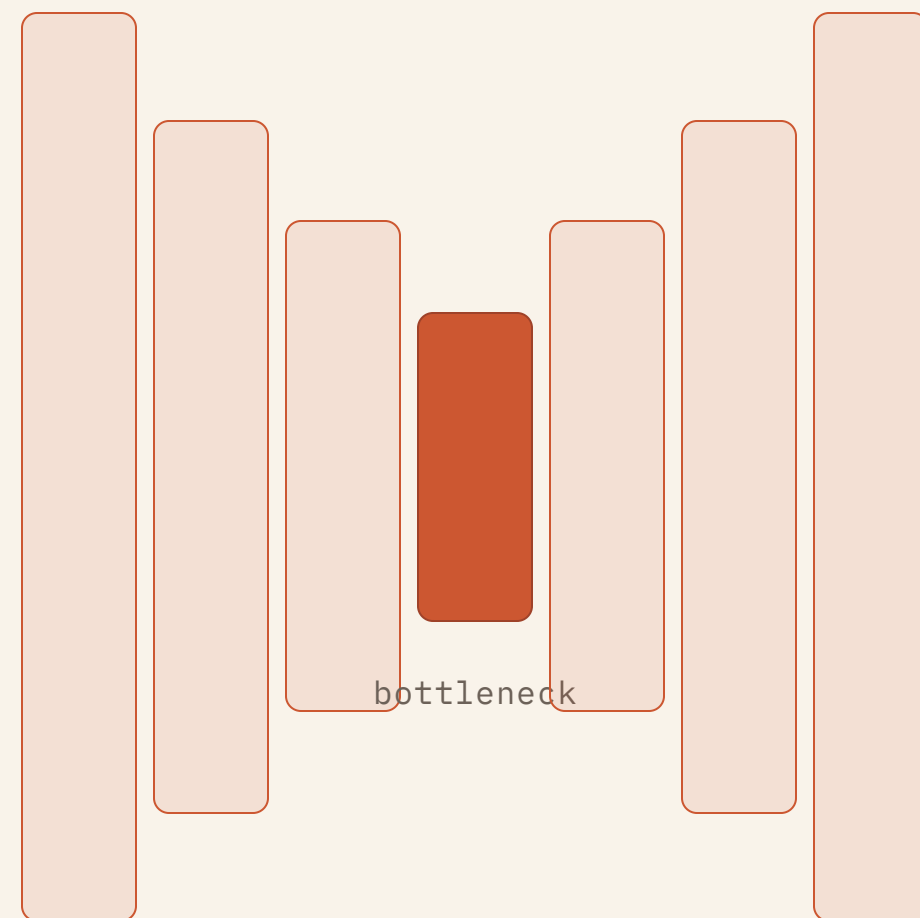
- ✗ **Low contrast** — a grey shirt on a grey wall is one number to the algorithm; the boundary vanishes.
- ✗ **Shadows & lighting** — produce strong edges that are *not* object boundaries.
- ✗ **Texture & clutter** — a leopard's spots fracture into dozens of false regions.

THE ROOT CAUSE

Classical methods have **no semantic understanding**. They see *numbers*, not objects.

To handle the real world's variation, we need a system that **learns** what objects look like — from data. Enter the neural network.

The encoder-decoder idea



Encoder shrinks the image while deepening understanding — edges → parts → objects (*what*).

Decoder upsamples back to full resolution, painting that understanding onto a pixel grid (*where*).

OUTPUT: A FULL-RESOLUTION MASK

Encoder — compress meaning

Decoder — restore location

From FCN to U-Net

2015 • THE BREAKTHROUGH

Fully Convolutional Network

Strip the dense classifier off a classification net so it outputs a **spatial grid** of predictions, not one label. Accepts any image size.

Limitation: upsampling from a tiny bottleneck gives *blurry* boundaries.

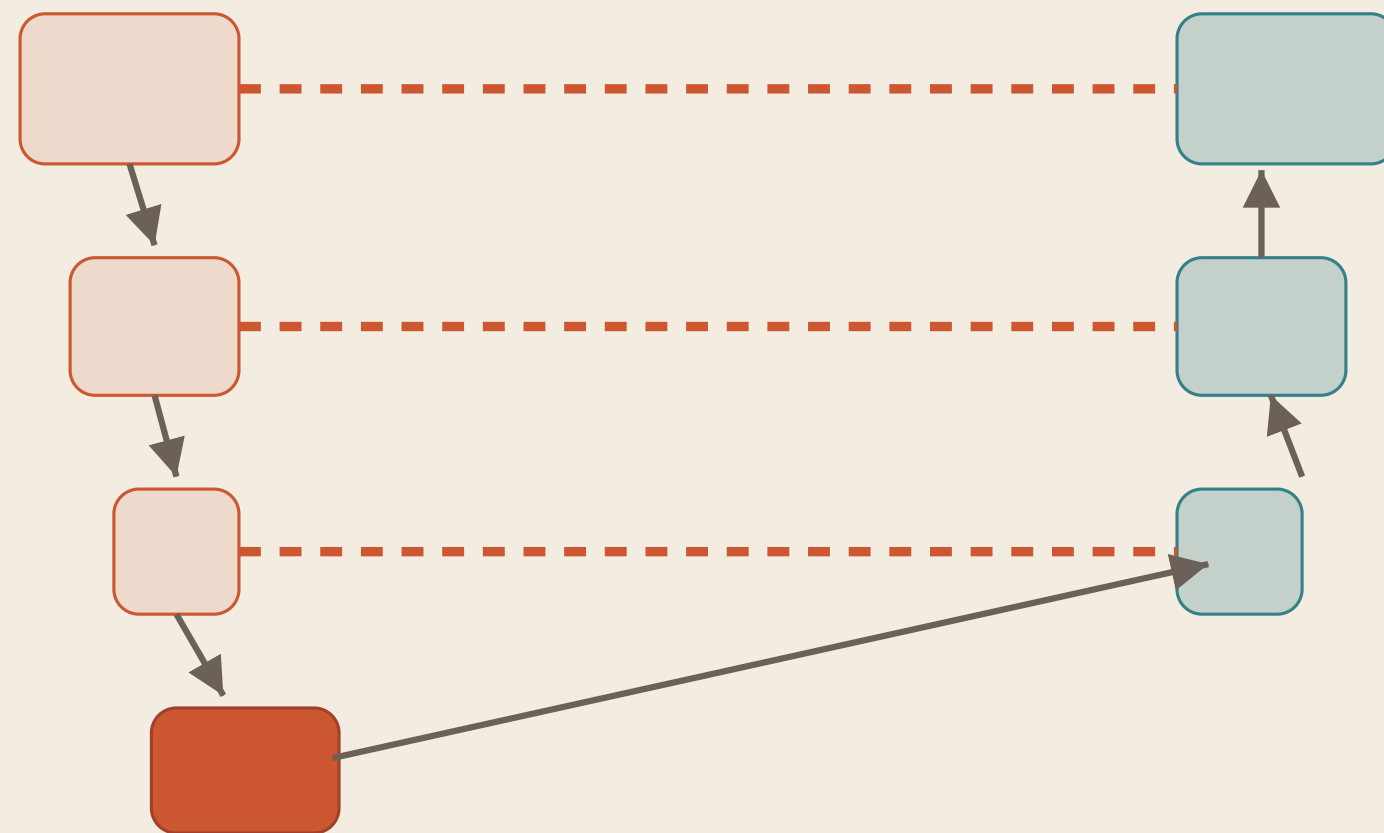
2015 • THE WORKHORSE

U-Net

A symmetric **U-shape** — encoder down, decoder up — born in biomedical microscopy. Adds **skip connections** to recover sharp detail.

Still the default for medical & scientific imaging a decade on.

Inside U-Net: skip connections



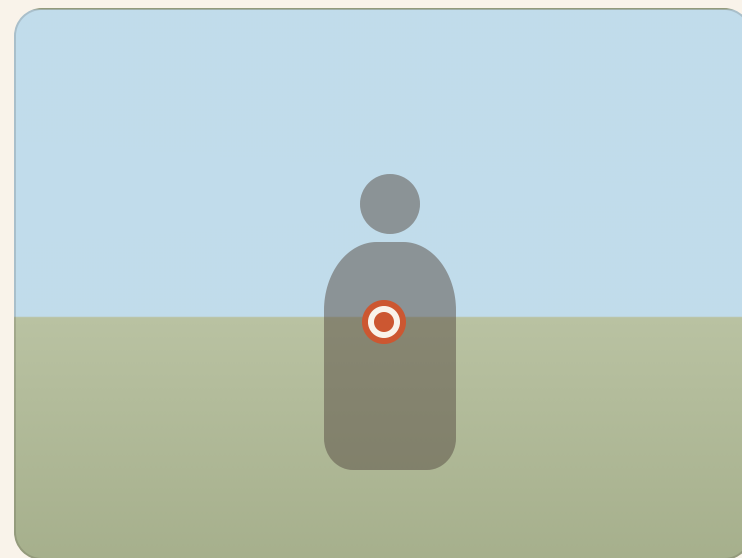
Each encoder level **copies its features across** to the matching decoder level.

The decoder fuses **what** (deep, abstract) with **where** (shallow, high-resolution) — giving masks that are both correct *and* crisp.

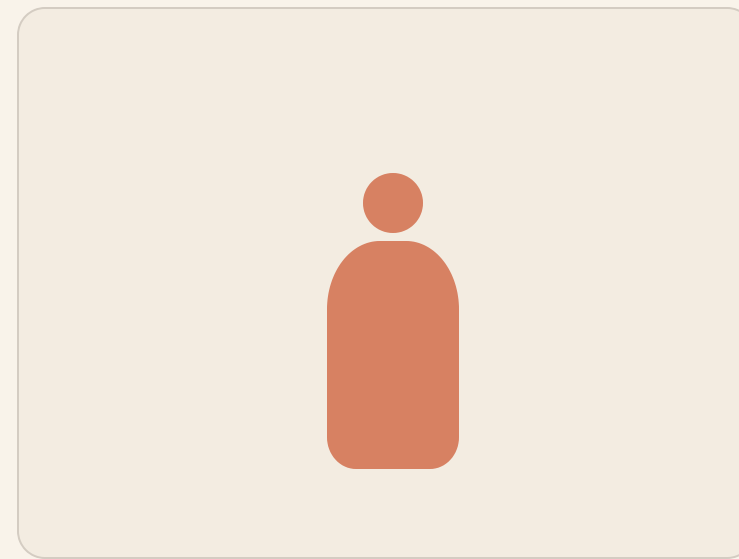
WHAT • FROM DEEP — WHERE • FROM SHALLOW

□ Encoder (down) □ Decoder (up) — Skip connection

Foundation models: Segment Anything (SAM)



Prompt • one click



Mask • any object

A **foundation model** — one network pre-trained on **1B+ masks** — that generalises to objects it was never explicitly taught.

Promptable: click a point or draw a box, get the mask back. Transforms data annotation & interactive tools.

The caveat: SAM segments anything but *names* nothing — it returns the mask, not the class.

A balanced landscape

CLASSICAL

Thresholding · edges

- + No data, no GPU, instant, interpretable.
- Breaks on real-world variation.

Use when: imaging is controlled.

CNN MODELS

U-Net · DeepLab

- + Accurate, efficient at inference.
- Needs labelled masks; fixed classes.

Use when: defined task + data.

FOUNDATION

SAM & successors

- + Generalises; cuts annotation cost.
- Large, compute-hungry, class-agnostic.

Use when: labelling / prototyping.

Complementary tools, **not a hierarchy** — match the method to your constraints: data, compute, and how controlled the imaging is.

The Python CV ecosystem

cv2 OpenCV

Classical image processing — read / write, colour conversion, resize, threshold, edges. The swiss-army knife.

torch PyTorch

The deep-learning engine — defines & runs neural nets on the GPU using **tensors** (n-dimensional arrays + autograd).

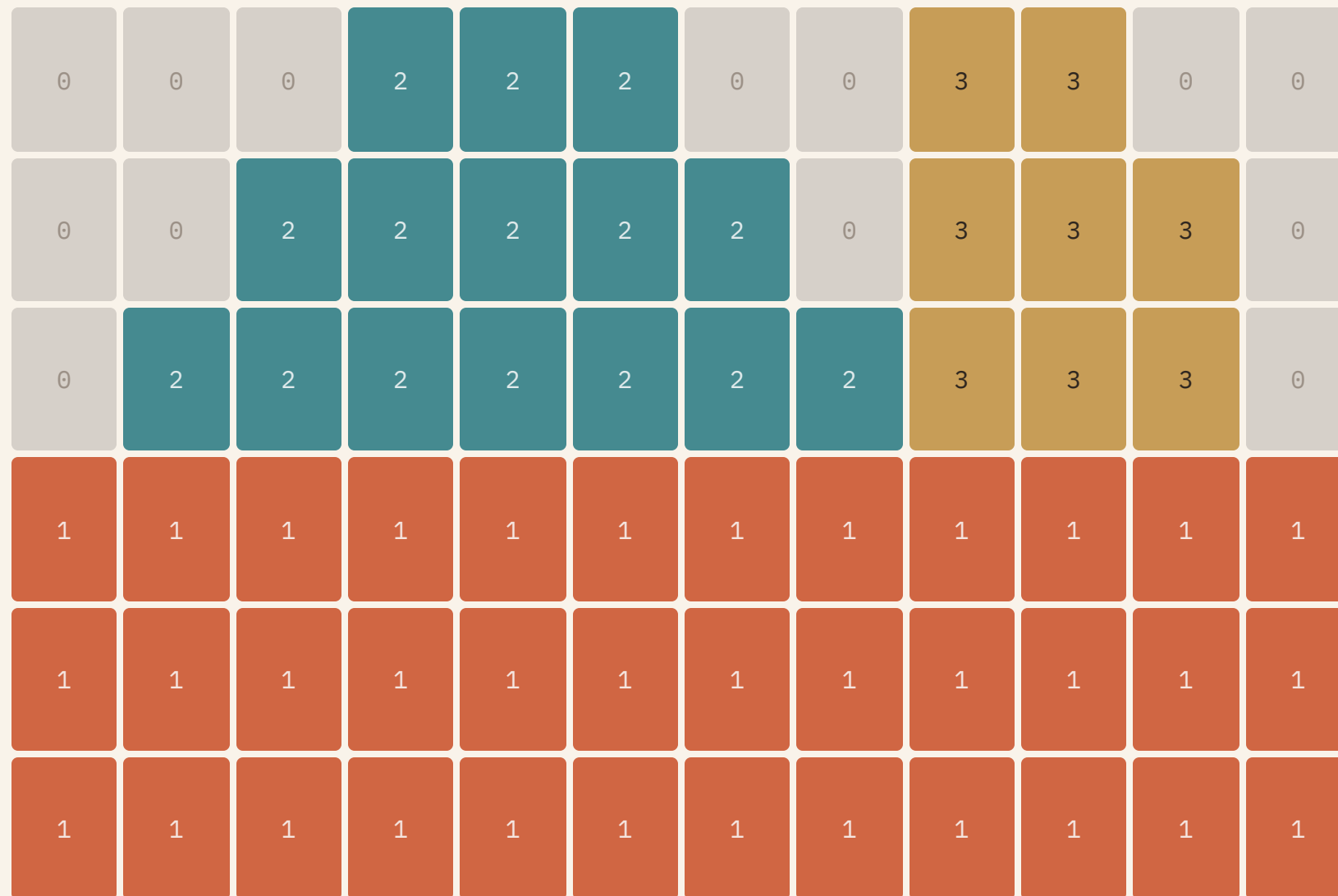
torchvision Torchvision

Ready-made **pre-trained models** (incl. DeepLabV3) & datasets.
Download a working segmenter in three lines.

A. Albumentations

Fast **data augmentation** — flips, rotations, colour jitter — transforming image *and* mask together, in sync.

Anatomy of a segmentation mask



0 · background 1 · road 2 · person 3 · sign

A mask is just a **2-D array of integers** — same H×W as the image — where each value is a **class index**.

Colours are only for *us*; the machine stores numbers. Convention: **0** = background.

Pixel **(*r*, *c*)** in the mask labels pixel **(*r*, *c*)** in the image — perfectly aligned.

The inference pipeline

STEP 01

Load & preprocess

Read the image; reshape into the model's expected format.

`cv2` • `transforms`

STEP 02

Load the model

→ Download DeepLabV3 with pre-trained weights; set eval mode.

`torchvision`

STEP 03

Inference + post-process

→ Forward pass, then argmax → a clean 2-D mask.

`torch.no_grad`

STEP 04

Visualise

→ Colour the mask; show it beside the original.

`matplotlib`

We are doing **inference, not training** — borrowing a finished model gives a real result in minutes.

Load & preprocess the image

```
import cv2, torch
from torchvision import transforms

# OpenCV loads as BGR — convert to RGB
img = cv2.imread("street.jpg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Match the model's training pipeline
preprocess = transforms.Compose([
    transforms.ToTensor(),           # HWC[0-255] → CHW[0-1]
    transforms.Normalize(           # ImageNet stats
        mean=[0.485, 0.456, 0.406],
        std =[0.229, 0.224, 0.225]),
])

# Add a batch dimension: CHW → 1·CHW
x = preprocess(img).unsqueeze(0)
```

- 1 BGR → RGB. OpenCV's colour order is reversed — convert or your colours are wrong.
- 2 `ToTensor` reorders axes to C×H×W and scales pixels to [0,1].
- 3 `Normalize` centres data with the **exact training statistics**.
- 4 `unsqueeze(0)` adds a batch dim — models always expect a batch.

Load a pre-trained model

```
import torch
from torchvision.models.segmentation import (
    deeplabv3_resnet50, DeepLabV3_ResNet50_Weights)

# 1 · Best available pre-trained weights
#     (COCO/VOC – 21 classes incl. person, car...)
weights = DeepLabV3_ResNet50_Weights.DEFAULT

# 2 · Build the net & load the weights (cached)
model = deeplabv3_resnet50(weights=weights)

# 3 · eval() turns OFF training behaviour
model.eval()

# 4 · Use the GPU if there is one
device = "cuda" if torch.cuda.is_available() else "cpu"
model = model.to(device)
```

- 1 Pre-trained weights. **DEFAULT** = best available, trained on 21 VOC classes.
- 2 Downloads once, then **caches** — no training on your machine.
- 3 **eval()** fixes batch-norm & dropout for inference. Easy to forget — don't.
- 4 Same code runs on **GPU or CPU**, just at different speeds.

Run inference & post-process

```
# 1 · No gradients at inference → faster, lighter
with torch.no_grad():
    out = model(x.to(device))["out"]
    # out shape → [1, 21, H, W] : 21 scores / pixel

# 2 · Per pixel, pick the highest-scoring class
#     argmax over the 21 channels → [H, W]
mask = out.argmax(dim=1)

# 3 · Drop batch dim, back to CPU, to NumPy
mask = mask.squeeze(0).cpu().numpy()

print(mask.shape, mask.dtype)
# → (H, W) int64 · each value is a class id
```

- 1 `no_grad()` switches off training bookkeeping — **memory & speed**.
- 2 Output is **21 logits per pixel** — a raw score for each class.
- 3 `argmax` picks the winning class → collapses 21 channels into **one mask**.
- 4 Result: a 2-D array of **class indices** — exactly our mask definition.

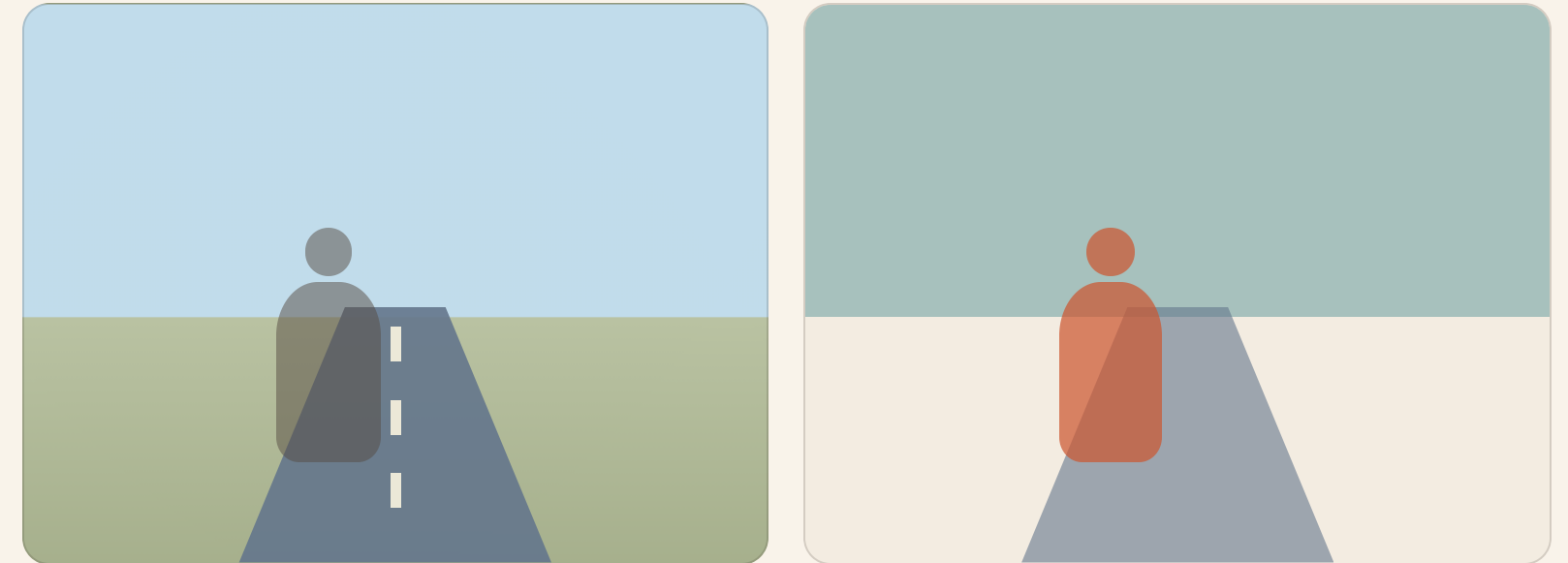
Visualise the output

```
import numpy as np
import matplotlib.pyplot as plt

# 1 · One colour per class index (0-20)
palette = np.random.randint(
    0, 255, (21, 3), dtype=np.uint8)

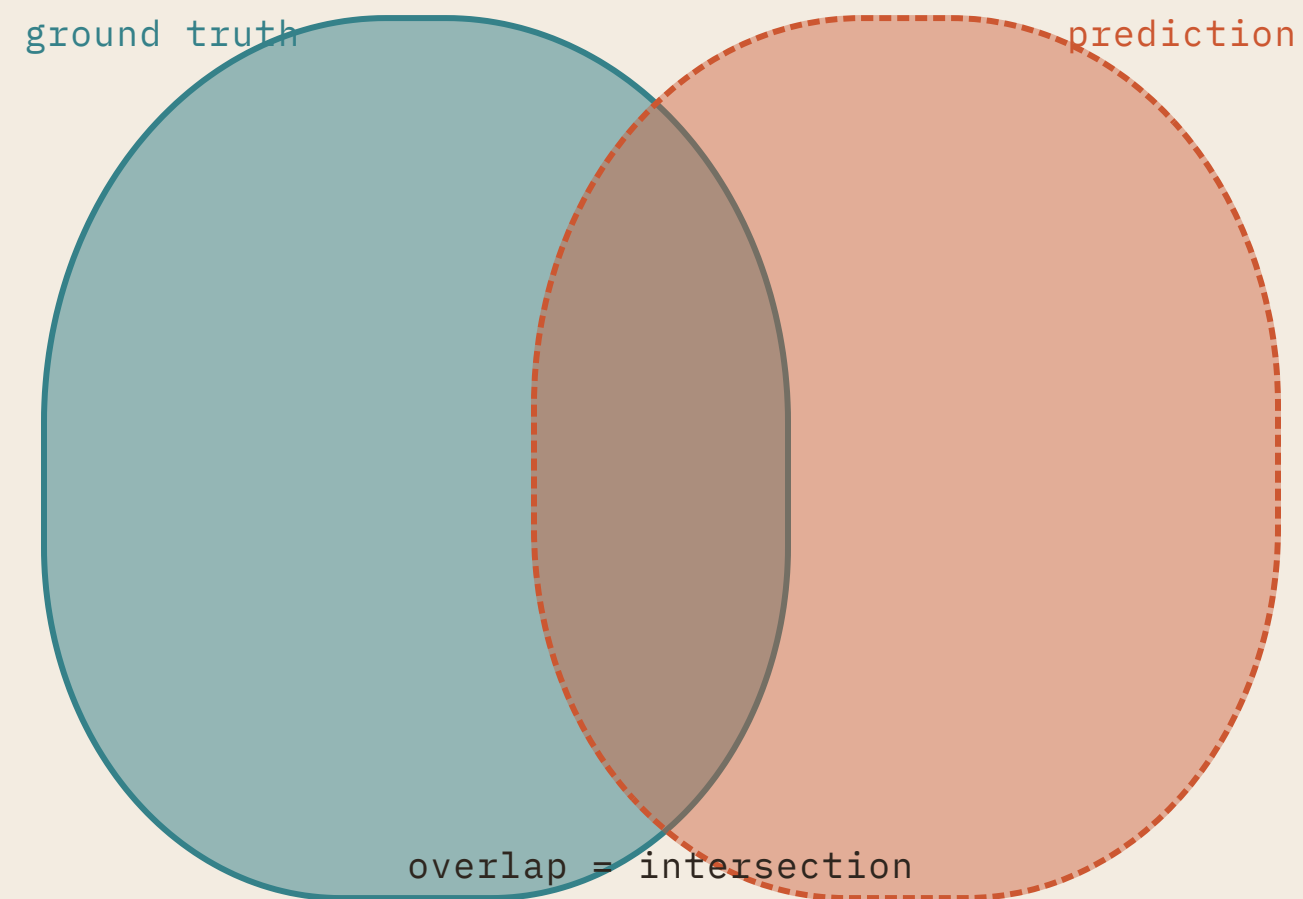
# 2 · Index map → colour image (H,W) → (H,W,3)
colour = palette[mask]

# 3 · Show input and segmentation together
fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].imshow(img); ax[0].set_title("Input")
ax[1].imshow(colour); ax[1].set_title("Segmentation")
for a in ax: a.axis("off")
plt.show()
```



★ `palette[mask]` maps every index to a colour in one line — raw pixels in, a meaningful map out.

Measuring quality: IoU



Compare the predicted mask against the human **ground truth**, pixel for pixel.

$$\text{IoU} = \frac{\text{Intersection}}{\text{Union}}$$

1.0 = perfect overlap · **0.0** = no overlap. Punishes both false positives *and* misses. Papers report **mean IoU** across classes.

Measuring quality: Dice score

The same idea as IoU — overlap — with a more forgiving formula. Common in **medical imaging**.

$$\text{Dice} = \frac{2 \times \text{Intersection}}{\text{Area}_{\text{pred}} + \text{Area}_{\text{truth}}}$$

Tracks IoU exactly (always moves together), but is **gentler on tiny objects** — a small tumour in a large scan. Often used as the training *loss* too.

RULE OF THUMB

- ↔ IoU & Dice **always agree** on which model is better.
- ≥ For the same mask, **Dice ≥ IoU**.
- ✗ **Never** compare an IoU number to a Dice number directly.

Common pitfalls & challenges

01 Class imbalance

99% background, 1% tumour → a model can score 99% *accuracy* and be useless. Use **IoU** / **Dice**, not raw accuracy.

02 Computational cost

Dense, full-resolution prediction is heavy — critical when a robot or surgical tool must run in **real time** on limited hardware.

03 Annotation cost

Hand-tracing masks takes minutes per image — the biggest practical bottleneck. The pain **SAM** aims to relieve.

04 Domain shift & edges

Daylight-trained models fail at night / in fog, and are fuzziest exactly at **object boundaries** — often the critical pixels.

Five things to remember

- 01 **Segmentation = pixel-level labelling.** The output is a mask — a 2-D array of class indices, same size as the image.
- 02 **It is the precise end of the spectrum** — label → box → mask — in three flavours: semantic, instance, panoptic.
- 03 **Evolution, not hierarchy:** classical → learned encoder-decoder (skip connections fuse *what* + *where*) → foundation models (SAM).
- 04 **In practice you do inference** with pre-trained models — load, model, infer, visualise — in a few dozen lines.
- 05 **Measure with IoU & Dice**, never raw accuracy; plan around imbalance, compute & annotation cost.

THANK YOU

Questions?

*Run today's pipeline on three of your own images
— bring the results next session.*

SEGMENTATION LABELS EVERY PIXEL

READ

U-Net & FCN papers · Segment Anything (SAM) paper

PRACTICE

Torchvision segmentation docs · official SAM notebooks · PyTorch & fast.ai tutorials

DATASETS

Pascal VOC · COCO · Cityscapes (driving) · Medical Segmentation Decathlon